

RENTipid Unified Multi-Login Authentication Module

Complete User, Business, Technical, Security and Operations Manual

Version v1.1.0

Document status: CLOSED / VERSION FROZEN

Frozen tag: `rentipid-unified-auth-v1.1.0-frozen`

Production runtime source SHA: `c0254631ea55030fd8e6c21ee73bc7a4563173ff`

Production deployment: `dp1_G2mNn7DAEJh4FMerSBcVhfnauZse`

Production site: <https://www.rentipid.com.ph>

Stable Preview: <https://preview.rentipid.com.ph>

Document revision: 1.0

Prepared: 2026-09-19

Audience: users, partners, prospects, support, engineering, security, operations, audit and governance

Owner: RENTipid

Classification: Internal publication baseline; no repository-specific confidentiality policy was located. Approve separately before external distribution.

Document Control

Field	Value
Document title	RENTipid Unified Multi-Login Authentication Module - Complete User, Business, Technical, Security and Operations Manual
Module	Unified Multi-Login Authentication Module
Version	v1.1.0
Release status	CLOSED / VERSION FROZEN
Production runtime source	<code>c0254631ea55030fd8e6c21ee73bc7a4563173ff</code>
Frozen tag	<code>rentipid-unified-auth-v1.1.0-frozen</code>

Field	Value
Document revision	1.0
Prepared date	2026-09-19
Audience	User, business, developer, security, operations and governance
Owner	RENTipid

Revision History

Revision	Date	Author/owner	Change	Status
1.0	2026-09-19	RENTipid documentation baseline	First complete coordinated manual derived from frozen source and release evidence	Published documentation successor

Document Purpose

This manual is the authoritative consolidated description of the accepted RENTipid Unified Multi-Login Authentication Module v1.1.0. It explains what customers use, what the business gains, how the implementation resolves identity, how security controls work and how operators support the service.

It does not replace source control, the frozen tag, provider-console configuration, incident authority or database change procedures. If this manual conflicts with the frozen implementation, the frozen implementation and accepted release evidence control and the documentation must be corrected.

Intended Audience

- Customers and marketplace providers who need sign-in and recovery guidance
- Business prospects and partners evaluating account continuity and security
- Developers maintaining the successor code line
- Operations and support teams monitoring or troubleshooting authentication
- Security reviewers and incident responders
- Product owners, auditors and release-governance authorities

How to Use This Manual

Start with Part I for the product model. Users can go directly to Part II. Business readers should use Part III. Developers and reviewers use Part IV and the Architecture/API/Security companion references. Operators use Part V. QA uses Part VI. Troubleshooting is in Part VII. API reference is in Part VIII. Security is in Part IX. Business, governance and release management are in Part X. Appendices are in Part XI, and the authoritative evidence register is in Part XII.

Document Conventions

- `Code style` identifies exact provider IDs, paths, models, variables, error codes, tags, commits and deployments.
- **NOTE** provides context.
- **IMPORTANT** identifies an action needed to preserve correct behavior.
- **WARNING** identifies security, privacy or availability risk.
- **TIP** provides practical guidance.
- "Implemented" means present in frozen source.
- "Configured" means release/runtime evidence records environment enablement.
- "Tested" means source tests or accepted execution evidence cover the behavior.
- "Documented" means described in accepted release/manual evidence.
- "Not implemented" means outside v1.1.0, even if discussed as a future possibility.

Important Security Notice

Email address similarity alone does not authorize identity linking. Same-email automatic OAuth linking is intentionally disabled. Do not enable or recommend `allowDangerousEmailAccountLinking`. Never place passwords, OTPs, OAuth codes/tokens, Apple ID tokens, session cookies, provider secrets, database credentials, Twilio tokens or Apple `.p8` material in logs, screenshots, documentation or support tickets.

Table of Contents

Front Matter

- [List of Figures](#)
- [List of Tables](#)
- [List of Diagrams](#)

Part I - Executive and Product Overview

1. [Executive summary](#)
2. [What is Unified Multi-Login](#)

3. [Why RENTipid uses multiple login methods](#)
4. [Problems solved and benefits](#)
5. [One user, many verified methods](#)
6. [Supported methods](#)
7. [Scope and non-goals](#)
8. [Release status](#)
9. [High-level architecture](#)
10. [Marketplace relationship](#)

Part II - User Guide

11. [Getting started and signing in](#)
12. [Provider instructions](#)
13. [Email and WhatsApp instructions](#)
14. [Account Security and Connected Login Methods](#)
15. [AccountLinkRequired](#)
16. [Recovery, logout and sessions](#)
17. [User privacy and safety](#)

Part III - Business Owner / Prospect Guide

18. [Business overview](#)
19. [Business benefit matrix](#)
20. [Account continuity and provider independence](#)
21. [Enterprise and marketplace readiness](#)

Part IV - Developer Reference

22. [Implementation architecture](#)
23. [Identity data model](#)
24. [Authentication algorithms](#)
25. [Provider implementations](#)
26. [Linking, collision and duplicate prevention](#)
27. [Sessions, RBAC and KYC](#)
28. [Configuration](#)

Part V - Operations

29. [Environment topology](#)
30. [Health and provider verification](#)
31. [Monitoring and support](#)
32. [Incident response](#)
33. [Rollback and reconciliation](#)

Part VI - Testing & QA

- 34. [Accepted verification evidence](#)
- 35. [Test strategy and matrix](#)
- 36. [Manual acceptance](#)

Part VII - Troubleshooting

- 37. [Diagnostic matrix](#)
- 38. [Provider-specific diagnostics](#)

Part VIII - API Reference

- 39. [API inventory](#)
- 40. [API security expectations](#)

Part IX - Security

- 41. [Security model and threat analysis](#)
- 42. [Cryptographic controls and key management](#)
- 43. [Multi-Factor Authentication \(MFA\) and session assurance](#)
- 44. [Audit events and logging hygiene](#)

Part X - Business / Governance / Release Management

- 45. [Ownership and change control](#)
- 46. [Mandatory release lifecycle](#)
- 47. [Evidence retention and successor releases](#)

Part XI - Appendices

- [Appendix A - Glossary](#)
- [Appendix B - Acronyms](#)
- [Appendix C - Supported Login Methods Matrix](#)
- [Appendix D - Provider Behavior Matrix](#)
- [Appendix E - Environment Matrix](#)
- [Appendix F - Environment Variable Reference](#)
- [Appendix G - Database Entity Reference](#)
- [Appendix H - API Route Reference](#)
- [Appendix I - Authentication Event Reference](#)
- [Appendix J - Error Code Reference](#)
- [Appendix K - User Troubleshooting Matrix](#)
- [Appendix L - Developer Troubleshooting Matrix](#)
- [Appendix M - Security Controls Matrix](#)
- [Appendix N - Threat Model](#)

- [Appendix O - Test Matrix](#)
- [Appendix P - Definition of Done](#)
- [Appendix Q - Release Lifecycle](#)
- [Appendix R - Deployment and Rollback Reference](#)
- [Appendix S - Support Escalation Matrix](#)
- [Appendix T - Business Prospect FAQ](#)
- [Appendix U - Developer FAQ](#)
- [Appendix V - User FAQ](#)
- [Appendix W - Release History](#)
- [Appendix X - Frozen Release Manifest](#)
- [Appendix Y - Architecture Diagrams](#)
- [Appendix Z - Document Change Log](#)

Part XII - Source & Evidence Register

- [Source paths](#)
- [Release documents](#)
- [Relevant commits](#)
- [Fact-classification summary](#)

List of Figures

Figure	Title	Location
Figure 1	Public Production sign-in gateway	Section 11

List of Tables

Major tables include document control, supported methods, benefit matrix, provider behavior, environment/configuration matrices, data entities, APIs, events, errors, troubleshooting, security controls, threats, tests, release lifecycle and evidence register.

List of Diagrams

1. High-level authentication architecture
2. One user/multiple identities
3. New-user authentication sequence
4. Returning linked-user login
5. Same-email new-provider flow
6. Explicit account-linking sequence
7. Provider collision sequence

- 8. Connected Login Methods workflow
- 9. Session flow
- 10. RBAC post-authentication flow
- 11. Local/Preview/Production topology
- 12. Database identity relationships

Reusable Mermaid sources are in [diagrams/](#).

Part I - Executive and Product Overview

1. Executive summary

RENTipid Unified Multi-Login v1.1.0 gives customers five supported ways to authenticate while preserving one internal RENTipid account. The methods are email/password, WhatsApp OTP, Google, Facebook and Apple.

The core design is identity continuity: a person can connect several verified methods to one permanent `User.id`. The internal user owns roles, permissions, KYC, profiles, listings, bookings, payments, ledger entries, reviews, history, sessions and security evidence. External providers do not own or assign these records.

The module rejects email-only automatic OAuth linking. A new provider that reports an email already associated with an account receives `ACCOUNT_LINK_REQUIRED`. The user must first authenticate the existing account and then connect the provider from Account Security. A provider subject already linked to another user is blocked with `IDENTITY_IN_USE`.

v1.1.0 is recorded as Production-deployed, completed, owner-accepted, closed and version-frozen. This manual describes that baseline without changing it.

2. What is Unified Multi-Login

Unified Multi-Login is the authentication layer that converts different proofs into one canonical user relationship.

```
flowchart TB
    G[Google subject] --> U[Permanent User U123]
    F[Facebook subject] --> U
    A[Apple subject] --> U
    E[EmailCredential] --> U
    P[PhoneIdentity] --> U
    U --> B[Business and marketplace records]
```

"Unified" does not mean unaudited merging. It means controlled resolution and linking around a permanent internal identity.

3. Why RENTipid uses multiple login methods

Rental-marketplace participants have different access needs. Social/provider sign-in can reduce entry friction. WhatsApp supports mobile-first users. Email/password provides a direct credential. Multiple connected methods can improve continuity when a provider is temporarily unavailable or a user changes device.

The choice also supports different user categories—renters, individual providers, business providers, operators, staff and administrators—without allowing the external provider to define their internal role.

4. Problems solved and benefits

Problems addressed

- Forced dependence on one credential style
- Duplicate accounts created by naive provider/email handling
- Loss of account continuity when an email changes or is hidden
- Unsafe same-email merging
- Provider identity reassignment
- Split business/KYC/transaction history
- Support uncertainty about linking and disconnecting

Benefits

- Familiar customer access options
- One customer record across connected methods
- Explicit anti-takeover linking
- Durable provider-subject ownership
- Provider-independent RBAC and business data
- Revocable sessions and security evidence
- Provider-specific failure isolation when alternatives are connected

5. One user, many verified methods

`User.id` is the permanent identity. Authentication identities are child records. Email metadata can help detect a possible existing account but cannot approve a link.

This distinction matters for Apple private relay, Facebook without email, Google email changes and corporate/customer address transitions. As long as a provider subject remains linked, the subject resolves to the same internal user. When a new subject appears, the secure linking rules apply.

6. Supported methods

Method	Public ID	Proof	Durable record	Notes
Email/password	<code>credential</code> s	Verified email credential and bcrypt comparison	<code>EmailCredential</code>	New credentials begin unverified
WhatsApp OTP	<code>phone-otp</code>	Twilio Verify WhatsApp challenge	<code>PhoneIdentity</code>	Five-minute default; five attempts; no SMS fallback

Method	Public ID	Proof	Durable record	Notes
Google	google	OAuth/OIDC callback	AuthProvide rIdentity	State, PKCE, nonce
Facebook	facebook	OAuth callback	AuthProvide rIdentity	State; email optional
Apple	apple	OAuth/OIDC form POST	AuthProvide rIdentity	State, PKCE, nonce; private relay aware

7. Scope and non-goals

In scope: provider sign-in, email credentials, WhatsApp verification, explicit identity linking, connected-method display and OAuth disconnect, sessions, recovery flows, MFA assurance integration, events, provider/config health and environment separation.

Not implemented in v1.1.0:

- Passkeys/WebAuthn
- Public SMS login
- Automatic same-email OAuth linking
- Provider-driven internal role assignment
- Universal offline authentication
- Automatic manual merging of duplicate users
- A business-performance guarantee or measured conversion uplift

8. Release status

Gate/evidence	Status
Production runtime source	Accepted at c025463...
Production deployment	dp1_G2mNn7DAEJh4FMerSBcVhfnauZse
Owner acceptance	PASS
Closure	PASS
Frozen tag	rentipid-unified-auth-v1.1.0-frozen
Preview isolation	Restored and separately evidenced after freeze; Production unchanged

9. High-level architecture

```
flowchart LR
    Client[Browser/PWA] --> Gateway[Sign-in gateway]
    Gateway --> Auth[NextAuth and UnifiedAuthService]
    Auth --> OAuth[OAuth[Google/Facebook/Apple]]
    Auth --> Twilio[Twilio Verify]
    Auth --> DB[(Prisma/PostgreSQL identity data)]
    DB --> User[Permanent User]
    User --> Session[JWT plus registry]
    Session --> RBAC[Internal authorization]
    RBAC --> App[RENTipid marketplace]
```

10. Marketplace relationship

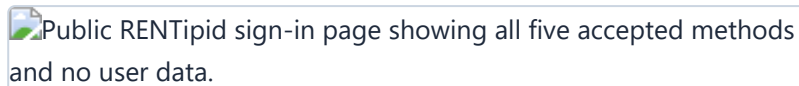
Authentication resolves a user; the marketplace authorizes that user. Renters retain bookings and reviews. Providers retain listings and verification. Businesses retain their business profile and related records. Operators/staff/admins retain only permissions granted inside RENTipid.

An external provider can be a sign-in mechanism for a privileged user, but its profile cannot create that privilege.

Part II - User Guide

11. Getting started and signing in

Open <https://www.rentipid.com.ph>, select Login and choose a method you already connected. New users can follow an onboarding method. Review terms/privacy before continuing.



Public RENTipid sign-in page showing all five accepted methods and no user data.

Figure 1. Public Production login page captured without authentication.

For full numbered instructions, see the [User Manual](#).

12. Provider instructions

Google

Select Continue with Google, choose the intended Google account and approve the provider flow. Returning linked subjects resolve the same RENTipid user. Same-email new subjects may require explicit linking.

Facebook

Select Continue with Facebook and authenticate at Facebook. Email may be absent. RENTipid uses the Facebook subject. Identity-in-use errors require support escalation.

Apple

Select Continue with Apple and authenticate. Share Email or Hide My Email can both work. RENTipid uses Apple's durable subject and handles the secure form POST callback.

13. Email and WhatsApp instructions

Email/password users enter their registered email and password. New credentials must be verified. Recovery tokens expire after 30 minutes and reset all sessions after success.

WhatsApp users enter a correctly formatted mobile number, request one code and submit it within the default five-minute window. Codes are one-time; users must never disclose them.

14. Account Security and Connected Login Methods

From Dashboard, open Account Security and Connected Login Methods. The panel shows Google, Facebook, Apple, Email & Password and WhatsApp OTP as Connected, Not connected or unavailable.

To add an OAuth method, select Connect, authenticate the provider and confirm Connected. To disconnect, confirm another tested method remains. The final viable method cannot be removed.

15. AccountLinkRequired

This state means the provider proof succeeded, but linking proof did not. It typically occurs when a new provider reports the same email as an existing RENTipid account.

Resolution:

1. Sign in through an existing connected method.
2. Open Account Security.
3. Select Connect for the new provider.
4. Authenticate the provider.
5. Confirm it is Connected.

This is expected protection, not a request to enable automatic merging.

16. Recovery, logout and sessions

Use another connected method when available. Email-password users can request a reset. External provider accounts must be recovered through the provider. If all methods are unavailable, use controlled RENTipid support without sharing secrets.

Log out through RENTipid rather than only closing the browser. Review active sessions and revoke unknown ones. Password reset revokes all sessions.

17. User privacy and safety

- Verify the official domain.
- Never share passwords, codes or cookies.
- Use a unique password and MFA where available.
- Store recovery codes privately.
- Check the selected account on shared devices.
- Review connected methods and sessions.
- Report unexpected linking/unlinking immediately.
- Understand that Apple relay email and absent Facebook email are normal supported states.

Part III - Business Owner / Prospect Guide

18. Business overview

Unified access is designed to lower unnecessary onboarding friction while preserving strong internal ownership. It offers customer convenience without making email the account key.

Operationally, one internal user reduces the need to reconcile separate bookings, profiles and provider records every time a person chooses another sign-in method. Security controls prevent convenience from becoming silent account merging.

19. Business benefit matrix

Capability	Customer benefit	Business benefit	Security benefit
Google	Familiar cross-device sign-in	Designed to reduce entry friction	Durable subject; state/PKCE/nonce
Apple	Private-relay-friendly sign-in	Serves Apple-oriented users	Durable subject and secure form POST
Facebook	Familiar sign-in even without email	Broadens choice	Subject is key; email optional
WhatsApp OTP	Passwordless mobile entry	Supports mobile-first customers	Expiry, limits, replay prevention
Email/password	Direct RENTipid credential	Provider-independent option	Hashing, verification, reset controls
Connected Methods	User-managed alternatives	Support clarity and continuity	Signed explicit linking
Collision protection	Clear ownership boundary	Reduces data-integrity disputes	Unique mapping and blocked reassignment
Internal RBAC	Consistent marketplace access	Protects operating model	Providers cannot assign roles

20. Account continuity and provider independence

The customer's marketplace history belongs to the internal user. Connecting Apple to a Google-backed account changes access options, not bookings or KYC ownership. If a provider is unavailable, another already connected method may continue to work.

The module does not guarantee that every customer has an alternate method. Partners should encourage appropriate account protection and support recovery processes.

21. Enterprise and marketplace readiness

Durable IDs, explicit linking, structured events, isolated environments and a gated release lifecycle support enterprise due diligence. The design can serve renters, providers, businesses and governed staff roles.

Geographic/language expansion still requires local legal, delivery, provider, localization and support review. No unsupported scale, availability or ROI figure is claimed.

Part IV - Developer Reference

22. Implementation architecture

The module uses Next.js 16 App Router, NextAuth 4.24.15, Prisma/PostgreSQL and a `UnifiedAuthService` with repository abstraction. Provider callbacks enter through the catch-all route (`/api/auth/[...nextauth]`) and are resolved to an internal user by service rules.

Key source files include `src/lib/auth.ts` , `src/lib/auth/unified/services.ts` , `repository.ts` , `identifiers.ts` , `oauth-consent.ts` , `oauth-link-intent.ts` , `session-registry.ts` , auth/account API routes, login UI, `ConnectedLoginMethods` and Prisma schema/migrations.

23. Identity data model

```
erDiagram
    User ||--o{ AuthProviderIdentity : owns
    User ||--o| EmailCredential : has
    User ||--o{ PhoneIdentity : owns
    User ||--o{ UserSession : has
    User ||--o{ AuthIdentityEvent : emits
    User ||--o{ AuthConsentReceipt : accepts
    User ||--o| UserProfile : has
    User ||--o| BusinessProfile : has
```

`AuthProviderIdentity` enforces a unique compound constraint on (`provider` , `provider_subject`) . `EmailCredential.normalized_email` and `PhoneIdentity.phone_e164` are unique. Challenges and rate limits support OTP controls. Consent, audit events, and session records provide governance, telemetry and revocation.

24. Authentication algorithms

Case 1 - linked provider

Resolve provider + subject, load canonical owner `User` , verify active account status, register session and complete sign-in.

Case 2 - new person

Validate profile and consent, confirm no existing provider identity or same-email account exists, create default canonical user and provider identity within a database transaction, then create a session.

Case 3 - same-email new provider

Do not link and do not create a duplicate account. Emit

`AUTH_ACCOUNT_LINK_REQUIRED`; require existing-account authentication followed by explicit connect via Account Security.

Case 4 - identity owned elsewhere

Block linking, emit `AUTH_IDENTITY_LINK_BLOCKED` with reason `IDENTITY_IN_USE`, preserve ownership integrity and escalate.

25. Provider implementations

Google: scope `openid email profile`; validates PKCE, state and nonce; provider subject is durable; validates issuer, audience, expiry and verified-email claims when present.

Facebook: state validation; durable subject; email optional; Meta callback registration strictly isolated per environment.

Apple: scope `name email`; `response_mode=form_post`; validates PKCE, state and nonce; secure `SameSite=None` transient cookies over HTTPS; durable subject; supports Apple private relay email metadata.

Email: normalized credential, bcrypt hashing, mandatory verification token flow via `POST`, generic recovery contract, hash-only expiring tokens.

WhatsApp: Twilio Verify integration, E.164 normalization, five-minute default expiry, five maximum attempts, persisted limits and atomic consumption. No SMS fallback.

26. Linking, collision and duplicate prevention

OAuth link intent is signed, valid for ten minutes, stored in an `HttpOnly` cookie, current-user-bound and provider-bound. Callback consumption distinguishes an explicit Connect action from an ordinary sign-in.

Database uniqueness and repository checks enforce single ownership. Same-owner linking is idempotent. Different-owner linking is strictly denied (`IDENTITY_IN_USE`). Unlinking verifies that at least one viable alternative login method remains.

Generic email/phone link and unlink APIs require AAL2 step-up assurance. Current OAuth Account Security actions use session plus signed intent and ownership safeguards; v1.1.0 does not claim AAL2 on every OAuth UI action.

27. Sessions, RBAC and KYC

JWT sessions carry user ID, role, status and an opaque session identifier.

`UserSession` stores only a hash of the session ID and supports 30-day maximum lifetime, last-seen tracking and revocation.

TOTP/recovery-code MFA can grant four-hour AAL2 step-up assurance bound to the active session. OAuth/phone login does not grant AAL2 automatically.

RBAC reads `User.role`; inactive account status fails closed. KYC, profile, business, booking, payment, and `FinanceLedger` relationships remain permanently bound to canonical `User.id`.

28. Configuration

Primary configuration variables are `NEXTAUTH_URL`, `NEXTAUTH_SECRET`, `AUTH_REFERENCE_HASH_SECRET`; provider feature flags, client IDs, and secrets; email/SMTP settings; WhatsApp/Twilio Verify settings; consent versions; database connection URLs; and MFA encryption key variables (`MFA_ENCRYPTION_KEY_ID`, `MFA_ENCRYPTION_KEY`, `RETIRED_FIELD_ENCRYPTION_KEYS`).

By source implementation design in `src/lib/auth/unified/config.ts`, `isAppleLoginDeferred()` defaults to `true` when `AUTH_APPLE_DEFERRED` is absent or empty. Therefore, Apple defaults to HIDDEN / DEFERRED on the public login gateway unless `AUTH_APPLE_DEFERRED` is explicitly configured to `false` (or `0`, `off`, `disabled`, `no`).

Only configuration names and purposes belong in documentation. Secret values reside exclusively in environment-specific secret management stores. See Appendix F and the companion Developer Manual for the complete table.

Part V - Operations

29. Environment topology

```
flowchart LR
    Local[LOCAL\nIsolated developer/test data]
    Preview[PREVIEW\nIndependent deployment and database]
    Production[PRODUCTION\nFrozen deployment and production database]
    Local -->|gated promotion| Preview
    Preview -->|accepted promotion| Production
```

Preview and Production must never share database instances, credentials, or live customer data. Post-freeze restoration evidence records independent Preview isolation without modifying the frozen Production runtime.

30. Health and provider verification

Read-only verification endpoints:

```
GET /api/health
GET /api/auth/providers
GET /api/auth/methods
```

In accepted environments, `/api/health` returns HTTP 200 with database connectivity healthy, and `/api/auth/providers` reports the five accepted provider IDs: `credentials`, `phone-otp`, `google`, `facebook`, and `apple`.

Provider registration does not prove end-to-end OAuth completion. Use designated test accounts and an approved acceptance runbook for end-to-end verification.

31. Monitoring and support

Monitor application health, database connectivity, provider registry changes, authentication failure rates, account-status denials, link-required and collision events, OTP rate-limit and replay failures, password recovery failures, MFA verification failures and session revocations.

Support telemetry collects environment, provider, timestamp, user agent/device, safe event references, and error codes. Support procedures must never collect passwords, OTPs, cookies, or session tokens.

32. Incident response

Unauthorized linking: preserve audit evidence, revoke active sessions, secure remaining methods, and verify durable subject ownership.

Secret exposure: contain exposure immediately, rotate secret in cloud vault, invalidate affected sessions/tokens, inspect logs for unauthorized usage, and document remediation.

Provider outage: keep alternative login methods available, confirm provider status and configuration, and never weaken or bypass authentication validation to restore access.

Environment crossover: halt promotion pipeline, preserve Production database, compare deployment IDs and database branch identifiers, correct configuration through controlled change, and reverify isolation.

33. Rollback and reconciliation

Application rollback deploys a known-good immutable deployment under documented incident authority. Database identity rollback favors forward repair or verified point-in-time recovery rather than manual deletion of identity constraints.

Identity reconciliation must be evidence-backed, auditable and reversible. It must preserve KYC, business profile, booking, payment, and **FinanceLedger** relationships, and never reassign an identity based on email similarity alone.

Part VI - Testing & QA

34. Accepted verification evidence

The Production completion report records typecheck PASS, targeted authentication regression PASS (8 suites, 85 tests) and canonical Production build PASS with Turbopack. Owner acceptance confirmed Google, Apple, and Facebook same-user linking, duplicate account prevention, and role/profile/KYC continuity.

The repository contains a broader current auth test inventory including legacy compatibility and post-acceptance regression files. Counts from source inventory must not be substituted for recorded accepted execution without active execution.

35. Test strategy and matrix

Testing covers service algorithms, route contracts, UI visibility, cookie security policies, display-email safety, ancillary email flows, and WhatsApp error containment.

Minimum mandatory scenarios include new/returning provider users, missing/private relay emails, Apple form POST handling, cross-provider same-email reconciliation, explicit linking, collision prevention, unlinking and last-method protection, credentials verification/reset, OTP expiration/replay/rate limits, provider outage tolerance, RBAC/KYC/marketplace continuity, logout/session revocation, and duplicate account prevention.

36. Manual acceptance

Use designated test accounts and non-production data in Preview. Record internal user continuity before and after connecting providers. Verify Account Security statuses and the absence of duplicate users. Never retain PII, tokens, cookies, or provider subjects in screenshots or test notes.

Production verification after deployment must be minimal, read-only, controlled, and evidence-based. Never use an acceptance exercise to create or alter live customer identity data.

Part VII - Troubleshooting

37. Diagnostic matrix

Symptom	Likely cause	Safe action	Escalation
Sign-in method not connected	Same-email provider not explicitly linked	Existing-method sign-in then Connect	L1/L2
Identity in use	Subject owned by another user	Stop; preserve evidence	Security/L3
Redirect URI mismatch	Provider console/environment mismatch	Correct exact registered callback	L3
Apple missing state cookie	Cross-site transient cookie not returned	Check HTTPS/Secure/SameSite/proxy; no bypass	L3
Provider absent	Disabled/deferred/unconfigured	Check public methods and env presence	L2/L3
OTP not received	Phone/delivery/rate issue	Verify number; wait; check Twilio safely	L1/L3
OTP expired/replayed	Challenge lifecycle	Start new challenge	L1
Invalid credentials	Credential proof failed	Generic retry/recovery	L1/L2
Cannot disconnect	Last viable method	Connect/test alternative	L1
Session returns to login	Expired/revoked/status/registry	Reauthenticate and inspect	L2/L3
Database unavailable	Health not ready	Restore dependency; no bypass	L3

38. Provider-specific diagnostics

Google: verify exact registered callback URI, provider registry status, PKCE/state/nonce validation, and OpenID claim integrity.

Facebook: verify Meta app callback configuration, accommodate absent email metadata, and escalate identity collision.

Apple: verify Services ID, form POST handling, and `SameSite=None` secure cookie behavior.

WhatsApp: verify Twilio Verify configuration, channel settings, challenge status, and rate limits.

Email: verify credential state, SMTP delivery health, and token lifetimes.

See the companion [Troubleshooting Guide](#) for full runbooks.

Part VIII - API Reference

39. API inventory

Framework-managed endpoints under `/api/auth/*` include providers, session, CSRF, sign-in, callbacks, and sign-out.

Application-level authentication endpoints include:

- `/api/auth/methods` (GET) — Public authentication method discovery
- `/api/auth/register` (POST) — Email/password account creation
- `/api/auth/otp` (POST) — WhatsApp OTP challenge start
- `/api/auth/oauth/intent` (POST) — Public OAuth consent token issuance
- `/api/auth/oauth/link-intent` (POST) — Authenticated OAuth linking intent issuance
- `/api/auth/link` (POST) — Credential/phone linking (Session + AAL2)
- `/api/auth/unlink` (POST) — Credential/phone unlinking (Session + AAL2)
- `/api/auth/logout` (POST) — Registry-aware session revocation
- `/api/auth/email-verification/resend` (POST) — Resend verification email
- `/api/auth/email-verification/verify` (POST) — Verify email address with one-time token
- `/api/auth/password-recovery` (POST) — Request password reset token
- `/api/auth/password-reset` (POST) — Execute password reset and revoke sessions
- `/api/auth/mfa/enroll` (POST) — Begin TOTP MFA enrollment
- `/api/auth/mfa/activate` (POST) — Complete TOTP MFA activation
- `/api/auth/mfa/verify` (POST) — Step-up verification for AAL2 assurance
- `/api/account/connected-methods` (GET, DELETE) — Manage OAuth provider identities
- `/api/account/sessions` (GET) — List active sessions
- `/api/account/sessions/{sessionId}` (DELETE) — Revoke specific session
- `/api/account/sessions/logout-others` (POST) — Revoke all other active sessions
- `/api/health` (GET) — Application and database readiness check

See the [API Reference](#) for full request/response schemas, error structures, and security expectations.

40. API security expectations

All APIs require same-origin HTTPS, framework CSRF protection, validated request payloads, and `Cache-Control: no-store` on sensitive responses. Email verification (`/api/auth/email-verification/verify`) is implemented strictly via `POST` accepting a JSON body `{ "token": "..."` } to avoid logging secrets in HTTP access logs. Sensitive error messages fail generic to prevent account enumeration.

Part IX - Security

41. Security model and threat analysis

Unified Multi-Login v1.1.0 operates under an explicit zero-trust identity boundary:

1. **Same-Email Account Takeover Defense:** Automatic linking based on email matching is disabled (`allowDangerousEmailAccountLinking: false`). When an incoming OAuth provider returns an email address associated with an existing account, the system halts authentication and emits `AUTH_ACCOUNT_LINK_REQUIRED` . Linking requires authenticating through an existing method and explicitly connecting the new provider.
2. **Provider Collision Protection:** Compound uniqueness on (`provider,` `provider_subject`) ensures an external identity can belong to at most one RENTipid user. Reassignment attempts trigger `AUTH_IDENTITY_LINK_BLOCKED` with reason `IDENTITY_IN_USE` .
3. **OAuth CSRF and Code Interception Defense:** State parameters and PKCE (Proof Key for Code Exchange) are enforced for Google and Apple. Nonce validation protects OpenID tokens.
4. **Cross-Site Callback Resilience:** Apple form POST callbacks utilize transient cookies configured with `SameSite=None; Secure` on HTTPS origins to prevent state loss across cross-site POST boundaries.
5. **OTP Abuse Defense:** WhatsApp OTP challenges enforce strict rate-limiting buckets (`AuthRateLimit`), 5-minute expiry windows, maximum 5 attempts, and atomic single-use consumption. SMS fallback is retired and blocked.

42. Cryptographic controls and key management

Cryptographic protections are managed centrally via `EnvironmentKeyProvider` (`src/lib/security/crypto/key-provider.ts`):

- **Field Encryption:** Sensitive user security assets (MFA TOTP secrets, recovery codes) are encrypted at rest using AES-256-GCM. The active encryption key is identified by `MFA_ENCRYPTION_KEY_ID` and loaded from `MFA_ENCRYPTION_KEY` (32-byte hex).
- **Key Rotation:** Zero-downtime key rotation is supported through `RETIRED_FIELD_ENCRYPTION_KEYS` , allowing historical ciphertext to be decrypted during rotation before re-encryption under the active key.
- **Reference Hashing:** User identifiers and provider subjects are pseudonymized in audit logs using HMAC-SHA256 reference hashes keyed by `AUTH_REFERENCE_HASH_SECRET` (falling back to `NEXTAUTH_SECRET`).
- **Token Hashing:** Email verification tokens and password reset tokens are stored exclusively as SHA-256 hashes in database tables. Raw token values are never

stored at rest.

- **Password Hashing:** Passwords are encrypted using Bcrypt (cost factor 10 on registration route, cost factor 12 via `BcryptPasswordHasher`).

43. Multi-Factor Authentication (MFA) and session assurance

RENTipid enforces a tiered session assurance model:

- **AAL1 (Standard Assurance):** Granted upon primary authentication via any supported login method.
- **AAL2 (Step-Up Assurance):** Required for privileged identity management actions (such as `/api/auth/link` and `/api/auth/unlink`). AAL2 is granted upon successful TOTP verification or recovery code submission via `POST /api/auth/mfa/verify`.
- **Assurance Binding & Expiry:** AAL2 assurance is recorded in `MfaSessionAssurance`, bound to the active session ID, and expires automatically after 4 hours (`MFA_SESSION_ASSURANCE_TTL_MS = 14400000`).
- **Assurance Invalidation:** Logging out, resetting passwords, or revoking sessions immediately clears associated AAL2 assurance records.

44. Audit events and logging hygiene

All authentication actions emit structured audit records with sanitized metadata.

Literal event families include:

- Lifecycle: `AUTH_LOGIN_SUCCEEDED`, `AUTH_LOGIN_FAILED`, `AUTH_ACCOUNT_STATUS_DENIED`, `AUTH_OAUTH_LOGIN_SUCCEEDED`, `AUTH_OAUTH_LOGIN_FAILED`, `AUTH_OAUTH_USER_CREATED`
- Linking: `AUTH_ACCOUNT_LINK_REQUIRED`, `AUTH_IDENTITY_LINK_BLOCKED` (reason `IDENTITY_IN_USE`)
- Phone OTP: `AUTH_PHONE_OTP_STARTED`, `AUTH_PHONE_OTP_VERIFIED`, `AUTH_PHONE_OTP_EXPIRED`, `AUTH_PHONE_OTP_REPLAY_DENIED`, `AUTH_PHONE_OTP_ATTEMPT_LIMITED`, `AUTH_PHONE_OTP_RATE_LIMITED`, `AUTH_PHONE_LOGIN_SUCCEEDED`, `AUTH_PHONE_USER_CREATED`
- Credentials: `AUTH_EMAIL_VERIFIED`, `AUTH_PASSWORD_RESET_COMPLETED`
- Sessions: `SESSION_CREATED`, `SESSION_REVOKED`, `SESSION_REVOKED_BY_USER`, `OTHER_SESSIONS_REVOKED`

Zero-Secret Logging Policy: Logging raw passwords, password hashes, OTP codes, OAuth codes, access/refresh tokens, Apple ID tokens, session cookies, client secrets, Twilio tokens, private keys, database connection strings, or full provider subjects is strictly prohibited across all loggers, telemetry, error handlers, and support tools.

Part X - Business, Governance and Release Management

45. Ownership and change control

RENTipid retains exclusive ownership of internal canonical user identities, role definitions, permission models, release evidence, and operational infrastructure. External providers own their authentication platforms and consumer recovery workflows. Users control their connected external accounts subject to provider terms.

Any modification to authentication architecture, schema, or runtime behavior requires approved successor work. Frozen release tags (`rentipid-unified-auth-v1.1.0-frozen`) and release commits are immutable and must never be amended or force-pushed. Secrets and provider console configurations are governed separately through secure cloud secret management.

46. Mandatory release lifecycle

The RENTipid engineering policy enforces a strict 14-gate promotion pipeline that must be traversed sequentially for all releases:

1. **CODE COMPLETE** — All code, tests, schemas, and documentation implemented.
2. **LOCAL FUNCTIONAL** — Feature runs locally without blocking runtime errors.
3. **LOCAL DATABASE MIGRATED** — Migrations applied and client synchronized.
4. **LOCAL REQUIRED DATA SEEDED/SYNCED** — System records, lookup values, and rules seeded.
5. **LOCAL ACCEPTANCE PASS** — Complete functional acceptance proven locally.
6. **PREVIEW MIGRATED** — Preview database migrated safely without affecting Production.
7. **PREVIEW ACCEPTANCE PASS** — Feature verified on deployed Preview environment.
8. **PRODUCTION-READY** — Security, database safety, rollback procedures, and documentation verified.
9. **PRODUCTION DEPLOYMENT** — Deployment to production infrastructure completed.
10. **PRODUCTION VERIFICATION** — Controlled, non-destructive health and provider verification.
11. **COMPLETED** — Module scope fully delivered and operational.
12. **OWNER ACCEPTED** — Formal sign-off and acceptance by module owner.
13. **CLOSED** — Engineering milestones sealed with closure report.
14. **VERSION FROZEN** — Release baseline immutable, tagged, and recorded in version frozen manifest.

Skipping any promotion gate is prohibited by policy.

47. Evidence retention and successor releases

Authoritative release evidence must be permanently retained: source commit SHAs, immutable tags, deployment IDs, migration timestamps, automated test results, build logs, provider client identifiers, acceptance reports, and environment isolation records. Credentials and sensitive secrets must never be included in retained evidence.

Successor releases must document deltas relative to baseline v1.1.0 (`rentipid-unified-auth-v1.1.0-frozen` , SHA `c0254631ea55030fd8e6c21ee73bc7a4563173ff`) and rerun all affected threat, schema, provider, linking, session, and regression suites.

Part XI - Appendices

Appendix A - Glossary

Term	Definition
Account	The permanent RENTipid <code>User</code> and its related records
Authentication identity	A verified method linked to a User
Canonical user	The permanent internal <code>User.id</code> owner
Connected method	A method whose durable identity/credential belongs to the current User
Durable subject	Provider-issued stable account identifier
Explicit linking	Authenticated process proving existing account and new provider control
Identity collision	A provider subject already owned by a different User
Provider metadata	Email/name/avatar supplied by an external provider, not ownership key
Synthetic email	Internal reserved identifier for provider/phone-only User rows

Appendix B - Acronyms

Acronym	Meaning
AAL2	Authentication Assurance Level 2 in the session step-up subsystem
CSRF	Cross-Site Request Forgery
ERD	Entity Relationship Diagram
HMAC	Hash-based Message Authentication Code
KYC	Know Your Customer
MFA	Multi-Factor Authentication
OIDC	OpenID Connect
OTP	One-Time Password/code
PKCE	Proof Key for Code Exchange

Acronym	Meaning
RBAC	Role-Based Access Control
PWA	Progressive Web Application

Appendix C - Supported Login Methods Matrix

Method	Accepted	Public	Can create user	Can return same user	Current direct Connected UI
Email/pass word	Yes	Yes	Yes	Yes	Displayed; credential flows separate
WhatsApp OTP	Yes	Yes	Yes	Yes	Displayed; verification flow separate
Google	Yes	Yes	Yes	Yes	Connect/di sconnect
Facebook	Yes	Yes	Yes	Yes	Connect/di sconnect
Apple	Yes	Yes	Yes	Yes	Connect/di sconnect
SMS	No - retired	No	Not a current public method	Legacy compatibility only	No

Appendix D - Provider Behavior Matrix

Provider	Durable key	Email behavior	Checks	Callback
Google	google + subject	Supplied email must be verified	state, PKCE, nonce	/api/auth/callback/google
Facebook	facebook + subject	Optional	state	/api/auth/callback/facebook

Provider	Durable key	Email behavior	Checks	Callback
Apple	apple + subject	Optional on return; can be private relay; supplied email verified	state, PKCE, nonce	/api/auth/callback/apple via form POST

Appendix E - Environment Matrix

Environment	Domain	Deployment/data rule	Provider target
Local	developer origin	Isolated local/test database; no Production secrets	As configured for development
Preview	preview.rentipid.com.ph	Independent Preview deployment and database	Five accepted IDs at preparation time
Production	www.rentipid.com.ph	Frozen deployment and Production database	Five accepted IDs

Appendix F - Environment Variable Reference

Category	Names	Secret?	Notes
Core auth	NEXTAUTH_URL , NEXTAUTH_SECRET , AUTH_REFERENCE_HASH_SECRET	URLs no; secrets yes	Strong isolated secrets required outside local
Google	AUTH_GOOGLE_ENABLED , GOOGLE_CLIENT_ID , GOOGLE_CLIENT_SECRET	Secret is sensitive	Exact callback per environment
Facebook	AUTH_FACEBOOK_ENABLED , FACEBOOK_CLIENT_ID , FACEBOOK_CLIENT_SECRET	Secret is sensitive	Meta registration per environment

Category	Names	Secret?	Notes
Apple	AUTH_APPLE_ENABLED , AUTH_APPLE_DEFERRED , APPLE_CLIENT_ID , APPLE_CLIENT_SECRET	Secret is sensitive	AUTH_APPLE_DEFERRED defaults to <code>true</code> (hidden); set <code>false</code> to expose
Email	AUTH_EMAIL_ENABLED , EMAIL_PROVIDER , EMAIL_FROM , SMTP_HOST , SMTP_PORT , SMTP_USER , SMTP_PASSWORD , SMTP_SECURE	Credentials sensitive	Frozen adapter is SMTP
Phone	AUTH_WHATSAPP_OTP_ENABLED , AUTH_SMS_OTP_ENABLED , TWILIO_ACCOUNT_SID , TWILIO_AUTH_TOKEN , TWILIO_VERIFY_SERVICE_SID	Twilio values sensitive	SMS false; Verify service used
Consent	RENTIPID_TERMS_VERSION , RENTIPID_PRIVACY_VERSION	No	Defaults identify unified v1.1 terms/privacy
Data/security	DATABASE_URL , DIRECT_URL , MFA_ENCRYPTION_KEY_ID , MFA_ENCRYPTION_KEY , RETIRED_FIELD_ENCRYPTION_KEYS	Keys/URLs sensitive	Environment isolation mandatory; AES-256 field encryption for MFA

Canonical Configuration Specification

Variable Name	Purpose	Secret?	Local	Preview	Production	Required?	Notes
<code>NEXTAUTH_URL</code>	Canonical application origin	No	<code>http://localhost:3000</code>	<code>https://preview.rentipid.com.ph</code>	<code>https://www.rentipid.com.ph</code>	Yes	Exact environment origin
<code>NEXTAUTH_SECRET</code>	JWT session signature & encryption	Yes	Local secret	Cloud Secret Store	Cloud Secret Store	Yes	Minimum 32-char high-entropy string
<code>AUTH_REFERENCE_HASH_SECRET</code>	HMAC reference hash key for audit logs	Yes	Local secret	Cloud Secret Store	Cloud Secret Store	Yes	Falls back to <code>NEXTAUTH_SECRET</code> if unset
<code>AUTH_GOOGLE_ENABLED</code>	Google OAuth feature flag	No	<code>true</code>	<code>true</code>	<code>true</code>	No	Defaults to <code>true</code>
<code>GOOGLE_CLIENT_ID</code>	Google OAuth client ID	No	Dev client ID	Preview client ID	Production client ID	Yes (for Google)	Exact callback registered in Google Cloud Console
<code>GOOGLE_CLIENT_SECRET</code>	Google OAuth client secret	Yes	Dev secret	Cloud Secret Store	Cloud Secret Store	Yes (for Google)	Never commit

Variable Name	Purpose	Secret?	Local	Preview	Production	Required?	Notes
<code>AUTH_FACEBOOK_ENABLED</code>	Facebook OAuth feature flag	No	<code>true</code>	<code>true</code>	<code>true</code>	No	Defaults to <code>true</code>
<code>FACEBOOK_CLIENT_ID</code>	Meta application ID	No	Dev App ID	Preview App ID	Production App ID	Yes (for Facebook)	Exact callback registered in Meta Developers
<code>FACEBOOK_CLIENT_SECRET</code>	Meta application secret	Yes	Dev secret	Cloud Secret Store	Cloud Secret Store	Yes (for Facebook)	Never commit
<code>AUTH_APPLE_ENABLED</code>	Apple OAuth feature flag	No	<code>true</code>	<code>true</code>	<code>true</code>	No	Defaults to <code>true</code>
<code>AUTH_APPLE_DEFERRED</code>	Controls public visibility of Apple login	No	<code>false</code> (to expose)	<code>false</code> (to expose)	<code>false</code> (to expose)	No	Defaults to <code>true</code> (HIDDEN/DEFERRED) when absent/empty; must be explicitly set to <code>false</code> to expose Apple

Variable Name	Purpose	Secret?	Local	Preview	Production	Required?	Notes
<code>APPLE_CLIENT_ID</code>	Apple Services ID	No	Dev Services ID	Preview Services ID	Production Services ID	Yes (for Apple)	Exact Services ID registered in Apple Developer
<code>APPLE_CLIENT_SECRET</code>	Apple generated ES256 client secret JWT	Yes	Dev JWT	Cloud Secret Store	Cloud Secret Store	Yes (for Apple)	Generated from <code>.p8</code> private key; never commit
<code>AUTH_EMAIL_ENABLED</code>	Email/password feature flag	No	<code>true</code>	<code>true</code>	<code>true</code>	No	Defaults to <code>true</code>
<code>EMAIL_PROVIDER</code>	Email delivery provider	No	<code>smtp</code>	<code>smtp</code>	<code>smtp</code>	No	Frozen adapter is SMTP
<code>EMAIL_FROM</code>	Transactional email sender address	No	Dev sender	Preview sender	Approved Production sender	Yes (for email)	Must be verified with mail provider
<code>SMTP_HOST</code>	SMTP server hostname	Sensitive	Dev SMTP host	Cloud Secret Store	Cloud Secret Store	Yes (for email)	Operational transport
<code>SMTP_PORT</code>	SMTP port	No	<code>587 / 465</code>	<code>587 / 465</code>	<code>587 / 465</code>	Yes (for email)	Parsed integer

Variable Name	Purpose	Secret?	Local	Preview	Production	Required?	Notes
<code>SMTP_USER</code>	SMTP authentication username	Sensitive	Dev user	Cloud Secret Store	Cloud Secret Store	Yes (for email)	Never commit
<code>SMTP_PASSWORD</code>	SMTP authentication password	Yes	Dev password	Cloud Secret Store	Cloud Secret Store	Yes (for email)	Never commit
<code>SMTP_SECURE</code>	SMTP TLS boolean	No	<code>false</code> / <code>true</code>	<code>false</code> / <code>true</code>	<code>false</code> / <code>true</code>	No	Boolean string
<code>AUTH_WHATSAPP_OTP_ENABLED</code>	WhatsApp OTP feature flag	No	<code>true</code>	<code>true</code>	<code>true</code>	No	Requires Twilio Verify configuration
<code>AUTH_SMS_OTP_ENABLED</code>	Legacy SMS feature flag	No	<code>false</code>	<code>false</code>	<code>false</code>	No	Retired; fails closed
<code>TWILIO_ACCOUNT_SID</code>	Twilio account identifier	Sensitive	Dev SID	Cloud Secret Store	Cloud Secret Store	Yes (for WhatsApp)	Account identifier
<code>TWILIO_AUTH_TOKEN</code>	Twilio API authentication token	Yes	Dev token	Cloud Secret Store	Cloud Secret Store	Yes (for WhatsApp)	Never commit
<code>TWILIO_VERIFY_SERVICE_SID</code>	Twilio Verify service identifier	Sensitive	Dev Verify SID	Cloud Secret Store	Cloud Secret Store	Yes (for WhatsApp)	Verify service container ID

Variable Name	Purpose	Secret?	Local	Preview	Production	Required?	Notes
<code>RENTIPID_TERMS_VERSION</code>	Terms document version string	No	<code>unified-multi-login-v1.1</code>	<code>unified-multi-login-v1.1</code>	<code>unified-multi-login-v1.1</code>	No	Default: <code>unified-multi-login-v1.1</code>
<code>RENTIPID_PRIVACY_VERSION</code>	Privacy policy version string	No	<code>unified-multi-login-v1.1</code>	<code>unified-multi-login-v1.1</code>	<code>unified-multi-login-v1.1</code>	No	Default: <code>unified-multi-login-v1.1</code>
<code>DATABASE_URL</code>	Runtime PostgreSQL connection string	Yes	Isolated dev DB	Independent Preview DB	Production DB	Yes	Connection pooling string; never cross environments
<code>DIRECT_URL</code>	Direct PostgreSQL migration connection	Yes	Isolated dev DB	Independent Preview DB	Production DB	Yes	Direct connection for Prisma migrations

Variable Name	Purpose	Secret?	Local	Preview	Production	Required?	Notes
<code>MFA_ENCRYPTION_KEY_ID</code>	Identifier for active field encryption key	No	local-key-1	Cloud Config / Store	Cloud Config / Store	Yes (for MFA)	Key ID/version metadata loaded via <code>EnvironmentKeyProvider</code> (<code>process.env.MFA_ENCRYPTION_KEY_ID</code>)
<code>MFA_ENCRYPTION_KEY</code>	AES-256 field encryption key for MFA secrets	Yes	32-byte hex string	Cloud Secret Store	Cloud Secret Store	Yes (for MFA)	Exactly 32 bytes (64 hex characters) paired with active <code>MFA_ENCRYPTION_KEY_ID</code>

Variable Name	Purpose	Secret?	Local	Preview	Production	Required?	Notes
RETIREDFIELD_ENCRYPTION_KEYS	Historical key decryption map for rotation	Yes	Optional / empty	Cloud Secret Store	Cloud Secret Store	No	JSON map ({ "keyId": "hexKey" }) for decrypting historical MFA secrets during key rotation
NODE_ENV	Runtime environment mode	No	development / test	production	production	Yes	Controls cookies and build optimizations

Appendix G - Database Entity Reference

Entity	Purpose	Critical constraint
User	Canonical account/business identity	Primary key; unique internal email
AuthProviderIdentity	OAuth identity ownership	Unique provider + subject
EmailCredential	Password credential	Unique user and normalized email
PhoneIdentity	Verified mobile identity	Unique E.164 phone
PhoneVerificationChallenge	OTP lifecycle	Status, attempts, expiry, consumption
AuthRateLimit	Persisted rate-limit buckets	Bucket/window semantics

Entity	Purpose	Critical constraint
AuthConsentReceipt	Terms/privacy consent evidence	User/version/time relationships
AuthIdentityEvent	Identity lifecycle audit	Safe provider/reference/reason
UserSession	Revocable session registry	Unique hashed session key
MfaSessionAssurance	Time-bounded AAL2	Bound to user/session hash
AuthenticationSecurityLog	Security telemetry	Sanitized metadata and retention

Appendix H - API Route Reference

Method	Path	Auth	Purpose
GET	/api/health	Public	Application/database readiness
GET	/api/auth/providers	Public	Runtime NextAuth providers
GET	/api/auth/methods	Public	Public method states
POST	/api/auth/register	Public	Email registration
POST	/api/auth/otp	Public	WhatsApp challenge start
POST	/api/auth/oauth/intent	Public	OAuth consent intent
POST	/api/auth/oauth/link-intent	Session	Explicit OAuth connect intent
GET/DELETE	/api/account/connected-methods	Session	List/disconnect OAuth method
POST	/api/auth/link	Session + AAL2	Generic credential/phone link
POST	/api/auth/unlink	Session + AAL2	Generic unlink
POST	/api/auth/logout	Session	Registry-aware logout

Method	Path	Auth	Purpose
POST	/api/auth/email-verification/resend	Public	Resend email verification token
POST	/api/auth/email-verification/verify	Public	Verify email address with one-time token
POST	/api/auth/password-recovery	Public	Request password reset token
POST	/api/auth/password-reset	Public	Reset password and revoke active sessions
POST	/api/auth/mfa/enroll	Session	Begin TOTP MFA enrollment
POST	/api/auth/mfa/activate	Session	Complete TOTP MFA activation
POST	/api/auth/mfa/verify	Session	Step-up verification for AAL2 assurance
GET	/api/account/sessions	Session	List sessions
DELETE	/api/account/sessions/{sessionId}	Session	Revoke another session
POST	/api/account/sessions/logout-others	Session	Revoke all other sessions

Appendix I - Authentication Event Reference

Event	Meaning
AUTH_ACCOUNT_LINK_REQUIRED	Same-email reconciliation stopped pending explicit proof
AUTH_IDENTITY_LINK_BLOCKED	Link denied, including IDENTITY_IN_USE
AUTH_OAUTH_USER_CREATED	New provider-backed internal user created
AUTH_OAUTH_LOGIN_SUCCEEDED	Existing provider identity resolved
AUTH_PHONE_OTP_STARTED/VERIFIED	WhatsApp challenge lifecycle success

Event	Meaning
AUTH_PHONE_OTP_EXPIRED/REPLAY_DENIED/ATTEMPT_LIMITED/RATE_LIMITED	OTP control outcome
AUTH_EMAIL_VERIFIED	Email credential verified
AUTH_PASSWORD_RESET_COMPLETED	Password changed and sessions revoked
SESSION_CREATED/REVOKED/REVOKED_BY_USER	Session registry lifecycle

Appendix J - Error Code Reference

Code	Meaning	Handling
ACCOUNT_LINK_REQUIRE_D	Existing account may match new provider email	Authenticate existing account, then connect
IDENTITY_IN_USE	Durable identity owned elsewhere	Block/escalate
LAST_SIGN_IN_METHOD	Unlink would strand account	Add/test another method
INVALID_OAUTH_PROFILE	Provider proof/profile invalid	Reject and diagnose callback
INVALID_CREDENTIALS	Credential proof failed	Generic retry/recovery
EMAIL_NOT_VERIFIED	Credential not verified	Verify/resend
INVALID_OTP	OTP challenge failed	Fresh challenge if safe
RATE_LIMITED	Threshold exceeded	Honor delay/investigate abuse
PROVIDER_UNAVAILABLE	Dependency unavailable	Alternative method/operations
METHOD_DISABLED	Method not enabled/configured	Alternative/config check
ACCOUNT_DISABLED	Account status denies access	Account-status process
CONSENT_REQUIRED	First-time OAuth consent missing	Restart gateway
RECENT_AUTH_REQUIRED	AAL2 required	MFA step-up

Appendix K - User Troubleshooting Matrix

Symptom	User action
Method not connected	Use existing method, then Account Security Connect
WhatsApp code missing	Verify number, wait, request one new code
Expired code	Start a fresh challenge
Forgot password	Use recovery; check newest link within 30 minutes
Cannot disconnect	Add/test another method first
Unknown session	Revoke it, secure methods, contact support if suspicious

Appendix L - Developer Troubleshooting Matrix

Symptom	Checks
OAuth callback failure	Exact origin/callback; registry; state/PKCE/nonce; clock; cookie policy
Apple state cookie missing	HTTPS, host/proxy, Secure, SameSite=None, 15-minute age
Facebook missing email	Accept subject; do not require/guess email
Identity collision	Ownership query, unique constraint, safe events; no reassignment
Session loop	callback normalization, registry, user status, session cookie, clock
Build sees root scratch file	<code>tsconfig.json</code> include/exclude and build remediation baseline

Appendix M - Security Controls Matrix

Control	Protects against	Evidence
No email auto-link	Same-email takeover	Service and connected-method tests
Provider+subject uniqueness	Identity collision	Schema/migration and tests
State/PKCE/nonce	OAuth CSRF/interception/replay	Config and Apple/security tests
Secure transient cookies	Apple form POST state loss	Cookie regression suite

Control	Protects against	Evidence
OTP expiry/attempt/rate/consume	Guessing/flood/replay	Service and route tests
Hash-only reset/verify tokens	Token disclosure at rest	Ancillary tests
Session registry	Stale/stolen session persistence	Session service/routes/events
Database RBAC	Provider privilege escalation	Service tests and acceptance
Sanitized events	Secret leakage	Audit-safety tests

Appendix N - Threat Model

Threat	Asset	Mitigation	Residual duty
Same-email takeover	Account ownership	Explicit dual proof	Support never merges by email
Provider collision	Identity ownership	Unique mapping and denial	Security investigation
OAuth CSRF	Callback integrity	State/CSRF/intents	Correct provider config
OTP abuse	Phone account access	Limits, expiry, attempts, replay prevention	Monitor provider/rate events
Credential stuffing	Email credential	Hashing/generic failures/rate infrastructure	Platform monitoring and response
Session theft	User access	HttpOnly cookie, registry, revocation	User/provider security
Role escalation	Privileged access	Internal role authority	Governance of role changes
Environment crossover	Customer data	Isolation and gates	Change control and verification

Appendix O - Test Matrix

Test ID	Scenario	Expected
O-01	New/returning Google	One User; returning same ID

Test ID	Scenario	Expected
O-02	New/returning Facebook	One User; missing email accepted
O-03	New/returning Apple	One User; relay metadata supported
O-04	Apple form POST	Secure transient cookies; state/PKCE/nonce retained
O-05	Cross-provider same email	AccountLinkRequired; no duplicate
O-06	Explicit provider connection	Same User owns multiple identities
O-07	Provider collision	Denied and audited
O-08	Unlink and last method	Secondary succeeds; last blocked
O-09	Email credential lifecycle	Verification/reset single-use and sessions revoked
O-10	OTP expiry/replay/limits	Denied/audited
O-11	RBAC/KYC/profile	Preserved
O-12	Booking/payment/ledger	Same canonical User relationships
O-13	Logout/revocation	Registry enforces revocation
O-14	Provider outage	Controlled failure; other methods independent

Appendix P - Definition of Done

For v1.1.0, done required implementation, migration, local/Preview/Production validation, provider acceptance, security regression, canonical build, Production verification, owner acceptance, closure and frozen evidence. A source-only change was not sufficient.

Appendix Q - Release Lifecycle

Gate	Required evidence
Code complete	Scoped source/schema/docs and review
Local functional	Working flows and tests
Local DB migrated	Migration applied and schema aligned
Local data seeded/synced	Required reference/test state

Gate	Required evidence
Local acceptance	Scenario evidence
Preview migrated	Isolated Preview database/deployment
Preview acceptance	End-to-end provider/linking behavior
Production-ready	Secrets/config/rollback/security approval
Production deployment/verification	Immutable deployment and health/provider checks
Completed/Accepted/Closed	Formal records and owner decision
Version frozen	Immutable tag/manifest

Appendix R - Deployment and Rollback Reference

Before deploy: secret scan, typecheck/tests/build, migration review, environment isolation, exact callbacks and rollback baseline. After deploy: health, provider IDs, existing-user login, link-required, explicit connect, collision, last-method, session and data continuity.

Rollback: use approved immutable deployment; preserve identity constraints; prefer reviewed forward fix/restore for database; restore provider configuration from secret management; rerun post-rollback verification.

Appendix S - Support Escalation Matrix

Condition	Level	Required action
Normal AccountLinkRequired	L1/L2	Educate and guide explicit connect
Provider outage/config issue	L3	Operations/provider investigation
Identity in use	Security/L3	Preserve evidence; no reassignment
Suspected takeover/session theft	Security	Revoke, secure, investigate
Duplicate account with business records	Security/Product/Data	Controlled reconciliation plan
Preview/Production crossover	Incident command	Stop, preserve Production, isolate and verify

Appendix T - Business Prospect FAQ

Does it automatically merge customers? No.

Can providers assign roles? No.

Does it replace KYC? No.

Can it reduce friction? It is designed to; no numerical outcome is claimed.

What if a provider fails? Existing alternative methods can provide continuity.

Is it accepted? v1.1.0 is Production-deployed, accepted, closed and frozen.

Appendix U - Developer FAQ

Canonical key? `User.id`.

OAuth key? Provider + provider subject.

Authoritative mapping? `AuthProviderIdentity`, not a NextAuth Account table.

Dangerous email linking? Not enabled.

Apple cookies? Secure SameSite=None only for required transient OAuth cookies on HTTPS.

Does OTP grant AAL2? No. TOTP/recovery-code verification grants AAL2.

Appendix V - User FAQ

Can Google and Apple reach the same account? Yes, after secure connection.

Why does same email not link automatically? Email alone is insufficient ownership proof.

Can I disconnect a method? Yes, if another usable method remains.

Is WhatsApp the same as SMS? No; SMS is retired.

Should support ask for a code? Never.

Appendix W - Release History

Date	Milestone	Evidence
2026-09-18	Apple transient-cookie remediation and unified linking lineage	Commits <code>41610f9...</code> , <code>9b21e5d...</code>
2026-09-19	Production runtime accepted	Source <code>c025463...</code> , deployment <code>dp1_G2m...</code>
2026-09-19	Owner acceptance, closure and freeze	Release reports and frozen tag
2026-09-19	Preview isolation restored without Production change	Post-freeze isolation report
2026-09-19	Complete manual suite prepared	This documentation successor

Appendix X - Frozen Release Manifest

Item	Value
Module	RENTipid Unified Multi-Login Authentication Module
Version	v1.1.0
Status	CLOSED / VERSION FROZEN
Tag	rentipid-unified-auth-v1.1.0-frozen
Tag commit	eea63c78f4c1cc11760bb3084d0ee9b84bac6e47
Tag object	ada8385ee3d0fc4e11fd710ea34f31dc60865a96
Production source	c0254631ea55030fd8e6c21ee73bc7a4563173ff
Production deployment	dp1_G2mNn7DAEJh4FMerSBcVhfnauZse
Providers	credentials , phone-otp , google , facebook , apple
Migration	20260827090000_unified_multi_login_auth_v1_1

Appendix Y - Architecture Diagrams

The 12 Mermaid sources in [diagrams/](#) cover high-level architecture, identity model, new/returning login, same-email stop, explicit linking, collision, Connected Methods, sessions, RBAC, environment topology and database relationships. They are derived from actual model and route names.

Appendix Z - Document Change Log

Revision	Change	Runtime impact
1.0	Initial complete manual suite for frozen v1.1.0	None; documentation only

Part XII - Source & Evidence Register

Source paths

Area	Paths reviewed
Auth framework	<code>src/lib/auth.ts</code> , <code>src/app/api/auth/[...nextauth]/route.ts</code>
Unified identity	<code>src/lib/auth/unified/config.ts</code> , <code>services.ts</code> , <code>repository.ts</code> , <code>identifiers.ts</code>
Intents/cookies	<code>oauth-consent.ts</code> , <code>oauth-link-intent.ts</code> , <code>anonymous-client.ts</code>
Credentials/recovery	<code>password.ts</code> , <code>ancillary.ts</code> , <code>email-delivery.ts</code> , register/recovery/verification routes
OTP	<code>phone-provider.ts</code> , <code>otp-route.ts</code> , OTP route, rate limiter
Sessions/MFA	<code>session-registry.ts</code> , session routes, <code>src/lib/security/auth/mfa-*</code>
UI	login page, ConnectedLoginMethods, Dashboard Security/Profile pages
Data	<code>prisma/schema.prisma</code> , unified identity and session migrations
Tests	nine auth-focused test files in <code>tests/auth/</code>

Release documents

Document	Use
<code>VERSION_FROZEN_MANIFEST.md</code>	Version/tag/provider/migration baseline
<code>UNIFIED_AUTH_CLOSURE_REPORT.md</code>	Closure architecture and policy evidence
<code>PRODUCTION_COMPLETION_REPORT.md</code>	Build/test/deployment/health evidence
<code>OWNER_ACCEPTANCE_REPORT.md</code>	Cross-provider same-user and integrity acceptance
<code>LOCAL_PRODUCTION_PARITY_REPORT.md</code>	Local parity/build evidence

Document	Use
<code>PREVIEW_ISOLATION_RESTORATION_REPO RT.md</code>	Current independent Preview topology
<code>AUTH_UNIFIED_LOGIN_PREVIEW_CHECKPO INT.md</code>	Historical implementation lineage; later frozen evidence prevails
<code>docs/modules/identity- authentication/reconciliation.md</code>	Reconciliation principles/context

Relevant commits

Commit	Relevance
<code>41610f9b536fd49a2457ceee87380612d0 0f8aea</code>	Apple form POST transient cookie remediation
<code>9b21e5d0203e610cfca8d2b5e27ef20174 a50389</code>	Controlled unified provider linking
<code>92328c85c8ff28011b8bdf4907939a560c edd09e</code>	Root scratch TypeScript/build scope remediation
<code>c0254631ea55030fd8e6c21ee73bc7a456 3173ff</code>	Accepted Production runtime source
<code>eea63c78f4c1cc11760bb3084d0ee9b84b ac6e47</code>	Frozen release manifest commit

Fact-classification summary

Classification	Examples in this manual
Implemented	Identity models, explicit linking, session registry, cookie policy, APIs
Configured	Five providers in accepted Production and current Preview registry
Tested	Auth regression behaviors and accepted release test/build evidence
Documented	Lifecycle, deployment IDs, owner acceptance and Preview isolation
Planned/not implemented	Passkeys, public SMS, automatic merging—identified as absent, not presented as current

End of Master Manual